Universidade de São Paulo – USP
Instituto de Matemática e Estatística – IME

Técnicas de Programação - MAC0216

RELATÓRIO TÉCNICO

Biblioteca Financeira em C

`Bibliotecas, MakeFile e Git`

Integrantes:
Isabela Neves Gonella - 17897490
Jorge Carneiro Silva Junior - 17836928
Maria Júlia Costa de Lima - 17974116
Millena Nunes Cupolillo - 15757513

Professor:
Helder May Nunes da Silva Oliveira

São Paulo – SP
Outubro de 2026

Sumário

1 Introdução **3**
1.1 Contextualização 3
1.2 Objetivo 3

2 Requisitos e especificações **5**

3 Arquitetura e organização do projeto **5**
3.1 Link do repositório no GitHub 6
3.2 Link do vídeo de demonstração 6
3.3 Link Apresentação de Slides 6

4 Implementação dos módulos **6**
4.1 Módulo de juros 7
4.2 Módulo de financiamento 7
4.3 Módulo de metas financeiras 8
4.4 Módulo de operações temporais 10

5 Sistema de compilação com Makefile **10**
5.1 Organização do processo de compilação 10
5.2 Alvos do Makefile 11
5.3 Dependências e compilação incremental 12

6 Geração e utilização das bibliotecas **12**
6.1 Biblioteca estática 12
6.2 Biblioteca dinâmica 13
6.3 Comparação entre ligação estática e dinâmica 13

7 Geração da documentação técnica **13**

8 Testes automatizados **14**
8.1 Estratégia de testes 14
8.2 Cobertura dos módulos 15
8.3 Exemplo de falha detectada e correção 15
8.4 Resultados dos testes 16

9 Controle de versão com Git **17**
9.1 Branches e integração das funcionalidades 17
9.2 Tag da versão final 18

10 Dificuldades e decisões de projeto **18**
10.1 Dificuldades encontradas 18
10.2 Decisões de projeto 19

11 Resultados obtidos **20**

12 Referências bibliográficas **21**

1 Introdução

1.1 Contextualização

A matemática financeira está presente em decisões cotidianas, como a escolha entre diferentes formas de financiamento, a avaliação do rendimento de uma aplicação, o planejamento para a conquista de metas de médio e longo prazo e o cálculo do valor de uma parcela paga antes ou depois do vencimento. Os cálculos envolvidos, como a capitalização de juros, a determinação do valor de parcelas, a contagem de dias entre datas e a estimativa de prazos, seguem fórmulas bem definidas, mas costumam ser reescritos em cada novo programa que os utiliza, o que favorece a duplicação de código e a introdução de erros.

Uma alternativa a esse cenário é reunir essas operações em uma biblioteca reutilizável. Em linguagem C, isso exige organizar o código em módulos com responsabilidades distintas, separando-se a interface pública, definida em arquivos de cabeçalho, dos detalhes de implementação, além de escolher a forma de distribuição da biblioteca, que pode ser estática, incorporada ao executável durante a linkagem, ou dinâmica, carregada apenas em tempo de execução.

Entretanto, à medida que o número de módulos cresce, a construção manual do projeto torna-se trabalhosa e suscetível a erros, já que é necessário compilar cada módulo separadamente, gerar as bibliotecas, compilar a aplicação e os testes e, a cada alteração, identificar quais componentes precisam ser reconstruídos. Soma-se a isso a necessidade de validar o comportamento das funções, em especial diante de entradas inválidas, como taxas negativas, datas inexistentes, prazos nulos ou dívidas que nunca serão quitadas, e de registrar a evolução do código de maneira rastreável.

Nesse contexto, o uso do `Make` para automatizar a construção do projeto, de testes automatizados para verificar o comportamento da biblioteca e do `Git` para o controle de versão estabelece um fluxo de desenvolvimento mais confiável e reproduzível.

A presente tarefa, descrita nesse relatório, busca atenuar tal problema através do desenvolvimento de uma biblioteca modular em C para cálculos de matemática financeira, acompanhada de uma aplicação cliente, de um sistema de compilação baseado em `Make` e de uma suíte de testes automatizados.

1.2 Objetivo

O objetivo desta tarefa é desenvolver, em linguagem C, uma biblioteca `financeira`, que reúne funções de matemática financeira organizadas em módulos funcionalmente distintos, e disponibilizá-la em suas versões estática (`libfinanceira.a`) e dinâmica (`libfinanceira.a`) para uso por uma aplicação cliente independente.

A biblioteca foi organizada nos seguintes módulos:

- `Juros`: calcula o montante de um valor aplicado uma única vez, com juros simples, compostos e contínuos;
- `Financiamento`: simula financiamentos com parcelas mensais fixas (sistema Price), calcula o valor financiável a partir de uma parcela e simula a amortização com pagamentos extras;
- `Metas`: apoia o planejamento de metas financeiras, calculando o percentual já atingido, o aporte mensal necessário e o prazo para alcançar o objetivo;
- `Tempo`: trata a dimensão temporal dos cálculos financeiros, determinando o número de dias entre duas datas, o número de meses necessários para quitar uma dívida com pagamentos fixos e o valor de uma parcela antecipada ou atrasada (valor presente e valor futuro).

Além da biblioteca, o projeto contempla os seguintes elementos:

- Uma aplicação cliente que utiliza exclusivamente a interface pública da biblioteca, definida nos arquivos de cabeçalho;
- A geração da biblioteca estática (`.a`) e da biblioteca dinâmica (`.so`), com a aplicação sendo compilada e executada com ambas;
- Um `Makefile` com os alvos `all`, `clean`, `run`, `test` e `install`, que representa as dependências entre os arquivos;
- Uma estratégia de compilação incremental, que evita a recompilação de arquivos não afetados por uma alteração;
- Testes automatizados para todas as funções públicas, cobrindo situações normais de uso, valores limite e entradas inválidas;
- O tratamento de erros de forma documentada e consistente, com a documentação das funções no formato Doxygen;
- O controle de versão com `Git`, incluindo o desenvolvimento de funcionalidades em branches separadas, a integração por meio de *merge* e a marcação da versão funcional com a tag `v1.0`.

2 Requisitos e especificações

A biblioteca disponibiliza operações financeiras organizadas em quatro áreas: Juros, Financiamento, Metas e Tempo. Suas funcionalidades incluem cálculos de capitalização simples, composta e contínua; simulação de financiamentos e amortizações; planejamento de metas financeiras; e operações envolvendo datas e atualização de parcelas.

As taxas de juros são representadas em formato decimal, de modo que uma taxa de 2% corresponde a 0,02. Nas operações dos módulos de Financiamento, Metas e Tempo, as taxas utilizadas são mensais. Os períodos financeiros discretos são expressos em meses, enquanto as funções de antecipação e atraso de parcelas utilizam intervalos em dias, considerando um mês comercial de 30 dias. O cálculo de juros contínuos admite tempo fracionário.

As funções devem validar os parâmetros recebidos de acordo com cada operação e sinalizar condições de erro por meio de valores de retorno ou códigos de status. Quando uma função recebe um ponteiro para armazenar o resultado, sua validade também deve ser verificada, quando aplicável.

Por fim, o projeto deve permitir a compilação automatizada, a geração de bibliotecas estática e dinâmica, a execução dos testes e a limpeza dos arquivos gerados, por meio de comandos definidos no `Makefile`.

3 Arquitetura e organização do projeto

A biblioteca foi estruturada de forma modular, separando os arquivos de implementação, as interfaces públicas, os testes automatizados e os artefatos gerados durante a compilação. Essa organização busca reduzir o acoplamento entre as diferentes partes do projeto e facilitar a manutenção e a expansão das funcionalidades.

de cabeçalho correspondentes estão organizados em `include/`. Os arquivos `.c` contêm a implementação das funções, ao passo que os arquivos `.h` declaram as interfaces disponibilizadas aos programas clientes, incluindo protótipos de funções, tipos de dados e, quando necessário, constantes relacionadas ao tratamento de erros.

Os testes automatizados são armazenados no diretório `tests/`, com arquivos específicos para cada módulo. O diretório `app/` contém o código-fonte da aplicação cliente utilizada para demonstrar a utilização da biblioteca. Durante a compilação, os arquivos objeto são gerados em `build/`, as bibliotecas resultantes são armazenadas em `lib/` e os executáveis são disponibilizados em `bin/`.

O diretório `docs/` reúne o relatório do projeto e a documentação técnica gerada pelo Doxygen, disponibilizada no subdiretório `docs/html/`. Essa documentação apresenta as informações extraídas do código-fonte, facilitando a consulta às funções, às suas interfaces e aos respectivos comentários.

Essa separação entre implementação e declaração permite que a aplicação utilize a biblioteca por meio dos contratos definidos nos arquivos .h, sem precisar conhecer os detalhes internos dos arquivos .c. Além de facilitar a manutenção, uma vez que as alterações na implementação de um módulo podem ser realizadas sem modificar o cliente, desde que sua interface pública seja preservada.

Os quatro módulos foram separados por responsabilidade funcional, e não apenas por conveniência de organização dos arquivos. As interfaces públicas também documentam os parâmetros, os resultados esperados e as condições de erro relevantes para cada operação.

Além disso, a comunicação entre os módulos ocorre por meio das interfaces disponibilizadas nos arquivos de cabeçalho. Um exemplo dessa integração está no módulo de Metas, que utiliza a função `juros_compostos`, disponibilizada pelo módulo de Juros, para obter o fator de capitalização necessário ao cálculo de aportes mensais. Dessa maneira, uma operação já implementada é reutilizada, evitando a duplicação desnecessária de cálculos.

Por fim, é válido ressaltar que a compilação e a geração dos artefatos são coordenadas pelo arquivo `Makefile`, responsável por definir as dependências entre arquivos, estabelecer as opções de compilação e automatizar a criação das bibliotecas e dos executáveis. Dessa forma, a estrutura do projeto separa as responsabilidades de implementação, disponibilização das interfaces, validação e construção do software.

3.1 Link do repositório no GitHub

https://github.com/isaneevess/Biblioteca-Financeira

3.2 Link do vídeo de demonstração

https://youtu.be/vHGPyLPIy6I?si=uNFT1Ql5xvdJ$_6db$

3.3 Link Apresentação de Slides

https://canva.link/hpyspoa5ti9e9mv

4 Implementação dos módulos

A biblioteca foi organizada em módulos independentes, cada um responsável por um conjunto específico de operações financeiras. Essa organização facilita a manutenção do código, a reutilização das funções e a realização de testes. As interfaces públicas são declaradas em arquivos de cabeçalho (`.h`), que especificam as funções disponíveis, seus parâmetros, valores de retorno e estruturas de dados utilizadas.

Como já descrito anteriormente, a implementação contempla os módulos de juros, financiamento, metas financeiras e operações relacionadas ao tempo. Além disso, foram

desenvolvidos testes automatizados para verificar o comportamento das funções em situações usuais, casos extremos e entradas inválidas.

4.1 Módulo de juros

O módulo de juros reúne funções para o cálculo de montantes segundo três modelos: juros simples, juros compostos e juros contínuos. As funções recebem os parâmetros necessários para cada modelo e retornam o montante calculado, considerando as restrições definidas na interface do módulo.

No regime de juros simples, o montante é calculado pela expressão

$$M = C(1 + it),$$

em que C representa o capital inicial, i corresponde à taxa de juros por período e t representa o número de períodos.

Para juros compostos, utiliza-se a expressão

$$M = C(1 + i)^t.$$

Nesse caso, os juros acumulados em cada período são incorporados ao capital para o cálculo dos períodos seguintes.

Já no regime de juros contínuos, o montante é obtido por

$$M = Ce^{it},$$

em que t pode assumir valores fracionários, permitindo representar intervalos de tempo não inteiros.

O módulo também define códigos de erro associados a parâmetros inválidos, como capital inicial negativo, taxa inferior ao limite permitido e período negativo. Essas verificações permitem identificar entradas incompatíveis com as condições estabelecidas para cada operação.

4.2 Módulo de financiamento

O módulo de financiamento implementa operações relacionadas ao cálculo de parcelas, à determinação do valor financiável e à simulação da amortização de uma dívida.

A função `simular_fin` recebe o valor total do bem, a entrada, a taxa de juros e a quantidade de parcelas. O valor financiado é obtido pela diferença entre o valor total e a entrada. A partir desse valor, calcula-se a prestação mensal, considerando a existência ou não de juros. O resultado é armazenado em uma *Structure* `Simulacao`, que contém o

valor da parcela, o total pago, o total de juros e o percentual de juros em relação ao valor financiado.

Quando a taxa é positiva, a prestação é calculada pela fórmula de uma série uniforme de pagamentos:

$$P = F\frac{i}{1-(1+i)^{-n}},$$

em que F representa o valor financiado, i é a taxa de juros por período e n é o número de parcelas. Quando a taxa é nula, utiliza-se $P = F/n$.

A função `valor_financiavel` realiza o cálculo inverso: a partir do valor da prestação, da taxa de juros e do número de parcelas, determina o valor que pode ser financiado. Para taxas positivas, utiliza-se

$$F = P\frac{1-(1+i)^{-n}}{i}.$$

Para taxa nula, o valor financiável corresponde ao produto entre a prestação e a quantidade de parcelas.

Por fim, a função `amortizacao` simula a evolução de uma dívida ao longo dos meses, considerando a incidência de juros sobre o saldo devedor e a realização de pagamentos regulares, acrescidos de eventuais pagamentos extras. A simulação é interrompida quando o saldo devedor se torna suficientemente pequeno. A *Structure* `Amortizacao` armazena a quantidade de parcelas necessárias e o total de juros acumulados.

As funções verificam condições como valores positivos para o financiamento e para a quantidade de parcelas, entrada inferior ao valor total e taxas não negativas. Na simulação de amortização, também se verifica se o pagamento é suficiente para superar os juros incidentes sobre a dívida no primeiro período. As operações retornam códigos de sucesso ou erro, permitindo que o programa que utiliza a biblioteca identifique entradas inválidas.

4.3 Módulo de metas financeiras

O módulo de metas financeiras disponibiliza funções para acompanhar o progresso de uma meta, calcular o aporte mensal necessário para alcançá-la e estimar o prazo necessário para atingir o valor desejado.

A função `meta_percentual_atingido` calcula a porcentagem da meta já acumulada. Para uma meta M e um valor atual V, o percentual é dado por

$$p = \frac{V}{M} \cdot 100.$$

Dessa forma, o resultado pode ser inferior, igual ou superior a 100%, dependendo do valor acumulado em relação à meta estabelecida.

A função `meta_aporte_mensal` determina o valor que deve ser depositado mensalmente para atingir uma meta em um número predeterminado de períodos. O cálculo

considera tanto o saldo inicial quanto o rendimento obtido pela aplicação desse saldo. Para uma taxa mensal positiva, o aporte é calculado por

$$A = \frac{M - V(1+i)^n}{\frac{(1+i)^n - 1}{i}},$$

em que M representa a meta, V é o valor inicial, i é a taxa mensal e n corresponde ao número de períodos. Quando a taxa é nula, o aporte necessário é calculado por $A = (M - V)/n$. Se o saldo inicial, após a incidência dos rendimentos, já for suficiente para atingir a meta, a função retorna zero.

A função `meta_prazo_necessario`, por sua vez, estima o tempo necessário para alcançar a meta a partir do saldo atual, do aporte mensal e da taxa de rendimento. Para uma taxa positiva, o cálculo utiliza logaritmos para determinar o número de períodos. Para uma taxa nula, o prazo é obtido pela razão entre o valor restante e o aporte mensal. O resultado pode ser fracionário, representando uma estimativa matemática do tempo necessário.

O módulo também distingue situações em que a meta é inalcançável sob as condições informadas e casos em que os parâmetros são inválidos. Para isso, são definidos códigos de retorno específicos, incluindo erros de parâmetro, de meta inalcançável e de ponteiro nulo. Essa diferenciação facilita o tratamento dos resultados pelo programa cliente.

Além disso, é importante destacar a consistência matemática entre as funções `meta_aporte_mensal` e `meta_prazo_necessario` é verificada por meio de testes de ida e volta. Ambas são baseadas na mesma equação de acumulação financeira, que considera o crescimento do saldo inicial e os rendimentos dos aportes mensais. Enquanto a primeira função determina o aporte necessário para atingir uma meta em um prazo fixado, a segunda estima o prazo necessário a partir de um aporte mensal informado.

Para verificar a coerência entre as implementações, inicialmente são definidos a meta financeira, o saldo inicial, a taxa de juros e um prazo desejado. Em seguida, calcula-se o aporte mensal por meio de `meta_aporte_mensal`. Esse resultado é utilizado como entrada para `meta_prazo_necessario`, mantendo os demais parâmetros constantes. Espera-se que o prazo recalculado seja aproximadamente igual ao prazo originalmente definido.

Esse procedimento permite verificar se as duas funções são matematicamente compatíveis, considerando uma tolerância para as diferenças decorrentes da representação numérica em ponto flutuante. A verificação é realizada tanto para taxas positivas quanto para taxa nula, caso em que as expressões de acumulação são simplificadas. Assim, os testes avaliam não apenas o funcionamento individual das funções, mas também a coerência entre diferentes operações disponibilizadas pelo módulo de metas financeiras.

4.4 Módulo de operações temporais

O módulo de operações temporais reúne funções para calcular intervalos entre datas, estimar o prazo de quitação de uma dívida e atualizar valores financeiros em função do tempo transcorrido.

A função `dias_entre_datas` recebe duas estruturas do tipo `Data`, compostas por dia, mês e ano, e calcula a diferença absoluta entre as datas em dias. Antes de efetuar o cálculo, verifica a validade das datas, considerando a quantidade de dias de cada mês e a ocorrência de anos bissextos. Caso alguma data seja inválida, a função retorna um código de erro.

A função `meses_para_quitar` simula a evolução de uma dívida submetida a juros mensais e pagamentos periódicos de valor fixo. A cada mês, os juros são incorporados ao saldo devedor e, em seguida, o pagamento é descontado. A simulação prossegue até a quitação da dívida, retornando a quantidade de meses necessária. Caso o pagamento não seja suficiente para superar os juros incidentes sobre a dívida, a função indica erro.

As funções `valor_antecipado` e `valor_atrasado` calculam, respectivamente, o valor presente de uma parcela paga antecipadamente e o valor atualizado de uma parcela paga com atraso. Ambas utilizam a taxa de juros e o intervalo de tempo em dias, adotando um mês comercial de 30 dias. As expressões utilizadas são

$$V_{\text{antecipado}} = \frac{P}{(1+i)^{d/30}}$$

e

$$V_{\text{atrasado}} = P(1+i)^{d/30},$$

em que P representa o valor original da parcela, i é a taxa de juros mensal e d corresponde à quantidade de dias.

As funções verificam a validade dos parâmetros, como o valor da parcela, a taxa de juros e o intervalo de tempo. Quando os dados fornecidos não atendem às condições estabelecidas, é retornado um valor de erro, conforme especificado nas interfaces do módulo.

5 Sistema de compilação com Makefile

5.1 Organização do processo de compilação

A compilação da biblioteca e da aplicação cliente é automatizada por meio de um arquivo `Makefile`, que centraliza os comandos necessários para construir os executáveis, gerar as bibliotecas, executar os testes e realizar tarefas de manutenção. Dessa forma, evita-se a necessidade de executar manualmente uma sequência de comandos do compilador a cada alteração no projeto.

O processo de compilação utiliza o GCC, com opções que habilitam avisos do compilador, especificam o padrão da linguagem C e permitem a geração de código adequado à construção da biblioteca dinâmica. A opção `-Iinclude` informa ao compilador o diretório em que estão localizados os cabeçalhos públicos, enquanto a opção `-lm` permite a ligação com a biblioteca matemática padrão, necessária para operações como exponenciação e logaritmos.

Além disso, o `Makefile` utiliza as opções `-MMD` e `-MP` para gerar informações de dependência entre os arquivos. Isso permite que o sistema identifique alterações em cabeçalhos utilizados pelos arquivos-fonte e determine quais componentes precisam ser recompilados.

A compilação é organizada em etapas: os arquivos-fonte são transformados em arquivos-objeto, esses objetos são reunidos para gerar as bibliotecas e, por fim, são utilizados na construção dos executáveis da aplicação cliente. Essa organização facilita a identificação de erros e permite aproveitar os arquivos que não precisam ser reconstruídos.

5.2 Alvos do Makefile

O arquivo `Makefile` disponibiliza alvos que automatizam as principais operações do projeto. Entre eles, destacam-se:

- `all`: reúne as principais etapas de construção, incluindo a geração das bibliotecas estática e dinâmica e dos executáveis da aplicação cliente;
- `clean`: remove os arquivos gerados durante a compilação, permitindo reconstruir o projeto a partir dos arquivos-fonte;
- `run`: executa a aplicação cliente compilada para utilizar a biblioteca dinâmica;
- `run-static`: executa a versão da aplicação cliente ligada à biblioteca estática;
- `test`: compila e executa os programas de teste dos módulos, apresentando os resultados das verificações;
- `doc`: executa o Doxygen a partir do arquivo `Doxyfile`, gerando a documentação técnica em HTML no diretório `docs/html/`.
- `clean-doc`: remove os diretórios `docs/html/`, eliminando os arquivos gerados pela documentação quando necessário.
- `install`: copia os cabeçalhos públicos e as bibliotecas para o diretório de instalação configurado.

A definição desses alvos simplifica o uso do projeto, pois cada operação pode ser solicitada diretamente pelo comando `make`, sem que o usuário precise conhecer todos os comandos internos de compilação.

O alvo `install` utiliza uma variável denominada `PREFIX` para definir o destino da instalação. Quando não há outra configuração, o diretório padrão corresponde a `$(HOME)/.local`, permitindo instalar os arquivos no diretório do usuário.

5.3 Dependências e compilação incremental

O sistema de compilação foi estruturado para evitar a reconstrução desnecessária de todos os componentes a cada execução do `make`. Para isso, os arquivos-objeto são armazenados no diretório `build/`, enquanto as dependências entre os arquivos são registradas automaticamente.

Quando um arquivo-fonte é alterado, o sistema verifica quais arquivos-objeto dependem dele e recompila os componentes afetados. De maneira semelhante, as dependências geradas permitem identificar alterações nos cabeçalhos utilizados pelos arquivos-fonte.

Após a atualização dos objetos necessários, os executáveis dependentes podem ser reconstruídos. Os componentes que continuam atualizados são preservados, reduzindo o trabalho realizado pelo compilador.

6 Geração e utilização das bibliotecas

6.1 Biblioteca estática

A biblioteca estática é gerada a partir dos arquivos-objeto produzidos durante a compilação dos módulos. No projeto, o arquivo resultante é armazenado no diretório `lib/` com a extensão `.a`.

A criação desse arquivo é realizada pela ferramenta `ar`, por meio do comando de arquivamento dos objetos compilados. A biblioteca resultante reúne as implementações dos módulos em um único arquivo, que pode ser utilizado na ligação de diferentes aplicações.

Para construir a versão estática da aplicação cliente, o arquivo executável é ligado à biblioteca estática gerada. Nesse processo, o ligador incorpora ao executável os trechos de código da biblioteca necessários à resolução das referências utilizadas pelo programa.

Uma consequência dessa abordagem é que o executável contém as implementações incorporadas durante a ligação, não dependendo do carregamento da biblioteca estática original durante sua execução. Entretanto, caso a implementação da biblioteca seja modificada, é necessário gerar novamente o executável para que ele incorpore as alterações.

No projeto, a geração da biblioteca estática e do respectivo executável é automatizada pelo alvo `all`, enquanto o alvo `run-static` permite executar a versão correspondente da aplicação cliente.

6.2 Biblioteca dinâmica

A biblioteca dinâmica é construída a partir dos mesmos arquivos-objeto utilizados na geração da biblioteca estática, mas emprega um processo de ligação diferente. O resultado é armazenado no diretório `lib/`, com a extensão `.so`.

Para gerar esse arquivo, o GCC é executado com a opção `-shared`. A opção `-fPIC`, utilizada durante a compilação dos arquivos-objeto, produz código adequado à construção de uma biblioteca compartilhada.

Na construção do executável correspondente, o ligador registra a dependência da biblioteca dinâmica em vez de incorporar integralmente suas implementações ao programa. Durante a execução, o sistema precisa localizar e carregar a biblioteca compartilhada para resolver essa dependência.

No projeto, a execução da aplicação cliente dinâmica é realizada pelo alvo `run`, que configura a variável de ambiente `LD_LIBRARY_PATH` para incluir o diretório `lib/`. Essa configuração permite que o carregador dinâmico localize a biblioteca gerada no próprio projeto.

A utilização da biblioteca dinâmica permite que diferentes executáveis compartilhem uma mesma implementação, além de possibilitar a atualização da biblioteca sem necessariamente reconstruir todos os executáveis que a utilizam, desde que a interface binária permaneça compatível.

6.3 Comparação entre ligação estática e dinâmica

As duas formas de ligação foram implementadas para permitir a comparação entre suas características e seus requisitos de execução. Embora ambas utilizem as mesmas funcionalidades da biblioteca financeira, elas diferem na maneira como o código da biblioteca é disponibilizado ao executável.

A ligação estática favorece a distribuição de executáveis que não dependam da biblioteca estática original no momento da execução. Por outro lado, a ligação dinâmica favorece o compartilhamento das implementações e pode simplificar a atualização da biblioteca quando sua interface permanece compatível.

No contexto deste projeto, a disponibilização das duas versões permite demonstrar que a biblioteca financeira pode ser reutilizada por aplicações clientes por meio de diferentes estratégias de ligação. Os alvos do `Makefile` automatizam a geração e a execução de ambas as versões.

7 Geração da documentação técnica

A documentação técnica do projeto é gerada automaticamente com o Doxygen, a partir dos comentários presentes no código-fonte e das informações contidas no arquivo

`README.md`. A configuração é definida no arquivo `Doxyfile`, que identifica os diretórios `include/` e `src/` como fontes para a documentação, além de utilizar o `README.md` como página inicial.

A configuração permite documentar o código em C, incluindo as funções, suas interfaces e os comentários associados. A documentação é gerada em formato HTML e armazenada em `docs/html/`, permitindo sua consulta por meio de um navegador. A opção `EXTRACT_ALL = YES` permite incluir na documentação os elementos identificados pelo Doxygen mesmo quando não possuem comentários explícitos, enquanto `HAVE_DOT = YES` habilita o suporte à geração de diagramas quando aplicável.

Para automatizar esse processo, foram definidos dois alvos no `Makefile`: `doc`, que executa o comando `doxygen Doxyfile`, e `clean-doc`, que remove os diretórios de documentação gerada, `docs/html/`. Assim, a documentação pode ser regenerada após alterações no código ou nos comentários, mantendo as informações disponíveis para consulta e manutenção da biblioteca.

8 Testes automatizados

8.1 Estratégia de testes

Os testes automatizados foram organizados para verificar o comportamento das funções públicas da biblioteca e identificar inconsistências nos resultados produzidos. Para isso, foram utilizados programas de teste separados por módulo, mantendo a organização dos testes alinhada à estrutura da biblioteca.

A estratégia contempla exemplos usuais de utilização, valores de fronteira e entradas inválidas. Nos testes de cálculos financeiros, os resultados obtidos são comparados com valores esperados, calculados a partir das fórmulas correspondentes. Como algumas operações utilizam números de ponto flutuante, as comparações numéricas consideram uma tolerância, evitando que pequenas diferenças de representação sejam interpretadas indevidamente como erros.

Os programas de teste utilizam verificações que informam se cada caso passou ou falhou. Quando uma verificação falha, o resultado é sinalizado, permitindo identificar a operação que apresentou comportamento diferente do esperado.

A execução dos testes é automatizada pelo alvo `test` do `Makefile`. Dessa forma, a compilação e a execução dos programas de teste podem ser realizadas por meio de um único comando: `make test`. Tal automatização torna o processo de verificação reproduzível e facilita a identificação de regressões após alterações nas implementações.

8.2 Cobertura dos módulos

Os testes foram distribuídos entre os módulos da biblioteca, considerando as operações disponibilizadas por cada um.

No módulo de juros, os testes verificam os cálculos de juros simples, compostos e contínuos, utilizando diferentes valores de capital, taxa e período. Também são examinadas situações como período nulo, taxa nula e parâmetros inválidos. As comparações são realizadas com uma tolerância numérica definida pelo programa de teste.

No módulo de metas financeiras, os testes verificam o percentual de uma meta já atingido, o cálculo de aportes periódicos e a estimativa do prazo necessário para alcançar determinado valor. Também são considerados casos como saldo inicial nulo, taxa de juros nula, meta já atingida e condições em que o objetivo não pode ser alcançado com os parâmetros fornecidos. Além disso, são testadas entradas inválidas e a consistência entre o cálculo do aporte e a estimativa do prazo.

No módulo de financiamento, os testes devem verificar a simulação do financiamento, o cálculo do valor financiável e a amortização de dívidas. A comparação entre simulações com e sem pagamentos adicionais permite avaliar se o programa representa adequadamente os efeitos desses pagamentos sobre o saldo devedor e os juros acumulados.

No módulo de operações temporais, os testes contemplam o cálculo da diferença entre datas, incluindo situações envolvendo anos bissextos e datas inválidas. Também são verificadas as estimativas de quitação de dívidas, bem como os cálculos de valores antecipados e atrasados, considerando diferentes parâmetros de entrada.

Essa organização permite localizar mais facilmente a origem de uma falha, uma vez que cada conjunto de testes está associado a um módulo específico. A cobertura efetivamente obtida deve ser avaliada a partir dos casos implementados nos arquivos de teste e dos resultados apresentados durante sua execução.

8.3 Exemplo de falha detectada e correção

Durante o desenvolvimento do módulo de juros, a função `juros_compostos` foi implementada com um erro de *off-by-one* no cálculo do expoente. A expressão utilizada era `pow(1.0 + taxa, periodos - 1)`, quando o correto seria `pow(1.0 + taxa, periodos)`. O erro foi registrado no commit `49d418e`.

Essa implementação fazia com que o montante calculado correspondesse a um período de capitalização a menos do que o solicitado. Por exemplo, para um capital inicial de R$ 1.000,00, uma taxa de 5% e três períodos, o resultado obtido era R$ 1.102,50, em vez do valor correto de R$ 1.157,63.

O teste automatizado de juros compostos com valores válidos, localizado em `tests/test_juros.c`, detectou a inconsistência ao comparar o resultado calculado com o valor esperado. A verificação foi sinalizada como falha, evidenciando que a implementação não

correspondia à fórmula matemática utilizada como referência.

A correção foi realizada no commit `8871de2`, com o ajuste do expoente para `periodos`. Após a alteração, os testes do módulo de juros passaram a ser aprovados, totalizando nove testes aprovados entre os nove executados.

Esse caso demonstra a importância dos testes automatizados durante o desenvolvimento. Uma falha aparentemente pequena na expressão matemática produzia um resultado incorreto, mas pôde ser identificada por meio da comparação com um valor esperado e corrigida antes de comprometer o uso da funcionalidade.

8.4 Resultados dos testes

A execução dos testes é centralizada no alvo `test` do `Makefile`, que compila os programas de teste e executa as verificações implementadas para os módulos da biblioteca. Os resultados são apresentados no terminal, permitindo identificar os casos aprovados e aqueles que falharam.

No caso documentado do módulo de juros, após a correção do erro no expoente da função `juros_compostos`, foram aprovados os nove testes executados. Esse resultado confirma a aprovação dos casos contemplados pelo conjunto de testes desse módulo, sem implicar, por si só, que todas as funcionalidades da biblioteca estejam livres de erros.

Para registrar o resultado global do projeto, devem ser considerados os resultados efetivamente obtidos na execução completa de `make test`. A quantidade total de testes aprovados, eventuais falhas remanescentes e os módulos correspondentes devem ser descritos de acordo com a saída real do comando.

Dessa forma, os testes constituem um mecanismo de verificação contínua do comportamento da biblioteca, auxiliando na identificação de erros, na validação das correções e na manutenção da consistência das funcionalidades implementadas.

9 Controle de versão com Git

O Git foi utilizado para acompanhar a evolução da Biblioteca Financeira e organizar o desenvolvimento de seus componentes. O histórico do repositório apresenta commits relacionados à criação das interfaces públicas, implementação dos módulos, desenvolvimento de testes, correções de erros, configuração do processo de compilação e integração da aplicação cliente.

A organização do trabalho utilizou branches específicas para os diferentes componentes do projeto. A branch `modJuros` concentrou alterações relacionadas ao módulo Juros; `modMetas`, ao módulo Metas; `financiamento`, ao módulo Financiamento; `modTempo`, ao módulo Tempo; `Makefille`, à construção do sistema de compilação; e `cliente`, à aplicação que demonstra a utilização da biblioteca. A branch `main` foi utilizada como linha principal de integração.

Essa estratégia permitiu desenvolver funcionalidades em linhas separadas e integrá-las posteriormente por meio de pull requests e merges. O histórico registra, por exemplo, a integração de alterações dos módulos de Financiamento, Tempo e Metas, além das mudanças relacionadas ao Makefile e à aplicação cliente.

O histórico também registra etapas de correção e refinamento da implementação. No módulo Juros, destacam-se a correção de um erro no expoente de `juros_compostos`, registrada no commit `8871de2`, e alterações posteriores no tratamento de erros. No módulo Metas, há commits relacionados à implementação de testes e ao ajuste da função `meta_aporte_mensal`. No módulo Tempo, o histórico registra a criação das funções e dos respectivos testes. No módulo Financiamento, aparecem commits relacionados à interface pública, à implementação das funções e aos testes do módulo.

Dessa forma, o histórico do Git documenta não apenas a criação dos arquivos, mas também a evolução incremental da biblioteca e a integração progressiva de seus componentes.

9.1 Branches e integração das funcionalidades

O desenvolvimento foi organizado em branches que correspondem às principais partes do projeto:

- `modJuros:` reuniu alterações no módulo Juros, incluindo a implementação inicial, os testes básicos e a correção do erro de *off-by-one* no expoente de `juros_compostos`. Também foram realizadas alterações posteriores no tratamento de erros.
- `modMetas:` reuniu alterações no módulo Metas, incluindo a implementação das funções, os testes automatizados e ajustes na função `meta_aporte_mensal`.
- `financiamento:` concentrou a criação da interface pública, a implementação das operações de financiamento e os testes associados.

- `modTempo:` concentrou a criação do módulo Tempo e o desenvolvimento de seus testes.
- `Makefille:` reuniu alterações relacionadas à construção do sistema de compilação, incluindo a criação do Makefile e a configuração de comandos para gerar e executar as aplicações.
- `cliente:` reuniu alterações na aplicação cliente, incluindo demonstrações dos módulos de Metas e Financiamento, ajustes no `main.c` e a utilização das bibliotecas estática e dinâmica.

O histórico registra integrações por meio de pull requests. Entre elas estão a integração de alterações do módulo de Financiamento, identificada pelo pull request #5; do módulo Tempo, pelo pull request #6; e do módulo Juros, pelo pull request #12. O módulo Metas também passou por diferentes integrações ao longo do desenvolvimento.

A integração mais recente corresponde ao commit `6dada23`, que ajusta o comando `clean-doc` no Makefile (`fix:ajuste no comando clean-doc no Makefile`). Esse commit está associado à tag `v1.0` e é o `HEAD` das branches `main` e `origin/main`.

9.2 Tag da versão final

A versão de entrega foi identificada pela tag `v1.0`, criada em 13 de outubro de 2026, com a descrição "Versão 1.0 para entrega'. A tag aponta para o commit `6dada23`.

Esse commit corrige o comando `clean-doc` do Makefile, responsável por remover a documentação gerada pelo Doxygen quando houver necessidade. As funcionalidades da aplicação cliente, incluindo a demonstração dos módulos e a utilização das bibliotecas estática e dinâmica, já haviam sido integradas em commits anteriores e continuam funcionando nessa versão.

10 Dificuldades e decisões de projeto

10.1 Dificuldades encontradas

Durante o desenvolvimento do projeto, algumas dificuldades exigiram decisões e ajustes por parte do grupo:

- **Originalidade e integração dos módulos:** por se tratar de uma biblioteca financeira, muitas das operações implementadas já existem em bibliotecas e sistemas semelhantes. Assim, uma das dificuldades foi encontrar uma forma de diferenciar o projeto, o que foi alcançado pela integração de quatro módulos completos: Juros,

Financiamento, Metas e Tempo. Essa organização permite reunir diferentes funcionalidades financeiras em uma única biblioteca, favorecendo o reaproveitamento de código e a utilização conjunta das operações.

- **Integração do `Makefile`:** centralizar a compilação dos módulos, a geração das bibliotecas estática e dinâmica, a construção da aplicação cliente e a execução dos testes exigiu cuidado com as dependências entre os arquivos. Também foi necessário organizar os diretórios e os alvos de compilação para que as alterações no código fossem consideradas corretamente, evitando recompilações desnecessárias.

- **Definição do modelo de testes automatizados:** outra dificuldade foi estabelecer uma estratégia de testes que permitisse verificar as funções de maneira sistemática e reproduzível. Foi necessário organizar os testes por módulo, definir entradas e resultados esperados e integrar sua execução ao processo de compilação. Essa estrutura contribuiu para identificar falhas nos cálculos e verificar as correções realizadas durante o desenvolvimento.

10.2 Decisões de projeto

A biblioteca foi organizada em quatro módulos funcionais: Juros, Financiamento, Metas e Tempo. Cada módulo possui responsabilidades específicas, com algumas dependências que permitem o reaproveitamento de funcionalidades. Além disso, a separação entre interface pública e implementação reduz o acoplamento entre a aplicação cliente e os detalhes internos da biblioteca.

No módulo de Juros, os períodos discretos de capitalização são representados por inteiros, enquanto o tempo na capitalização contínua utiliza o tipo `double`, permitindo valores fracionários. As taxas são expressas em formato decimal, e parâmetros inválidos são sinalizados pelo retorno `-1.0`. Também foram utilizadas as funções `pow()` e `exp()`, da biblioteca `math.h`, para realizar as operações matemáticas necessárias.

No módulo de Financiamento, foram utilizadas estruturas para agrupar os resultados das simulações e amortizações, tornando as interfaces das funções mais organizadas. Também foram implementadas validações dos parâmetros de entrada e dos ponteiros utilizados para armazenar resultados, quando aplicável.

No módulo de Metas, as funções permitem calcular o percentual atingido, o aporte mensal necessário e o prazo para alcançar um objetivo financeiro. Na função `meta_percentual_atingido`, valores negativos são rejeitados por serem incompatíveis com a interpretação financeira adotada. Além disso, foram definidos códigos de status específicos para distinguir diferentes condições de erro.

No módulo de Tempo, a estrutura `Data` agrupa as informações de dia, mês e ano. Para os cálculos de antecipação e atraso de parcelas, adotou-se o mês comercial de 30

dias, estabelecendo uma convenção uniforme para essas operações.

A automação do processo de compilação foi centralizada no `Makefile`, que reúne comandos para gerar as bibliotecas, compilar a aplicação cliente, executar os testes e limpar os arquivos gerados. Foram definidos os alvos `run` e `run-static`: o primeiro utiliza a biblioteca dinâmica e foi escolhido como padrão por permitir a atualização da implementação sem exigir, necessariamente, a recompilação dos programas clientes; o segundo permite executar a aplicação com a biblioteca estática e comparar as duas formas de vinculação.

Durante o desenvolvimento, foi identificado e corrigido um erro na função `juros_compostos`, relacionado ao cálculo do expoente utilizado na capitalização. A implementação inicial apresentava um erro de uma unidade no expoente, comprometendo o valor do montante calculado. A correção foi registrada no histórico de commits, permitindo acompanhar a evolução da implementação e documentar o processo de identificação e resolução da falha. Essa prática contribuiu para aumentar a confiabilidade da biblioteca e garantir maior consistência na versão final entregue.

Por fim, foram utilizados comentários e configurações compatíveis com o Doxygen para gerar automaticamente a documentação técnica. Essa escolha permite consultar as funções e suas interfaces em formato HTML e regenerar a documentação após alterações no código, facilitando a manutenção e a utilização da biblioteca.

11 Resultados obtidos

O projeto resultou no desenvolvimento de uma biblioteca financeira em C, organizada em quatro módulos: Juros, Financiamento, Metas e Tempo. A integração dessas funcionalidades disponibiliza, em uma única biblioteca, operações úteis para cálculos de juros, simulações de financiamentos e amortizações, planejamento de objetivos financeiros e ajustes de parcelas. A separação entre interfaces públicas e implementações também permite que as funcionalidades sejam utilizadas por diferentes aplicações clientes, sem que seja necessário conhecer os detalhes internos de cada módulo.

Um dos principais resultados foi a geração das versões estática e dinâmica da biblioteca. A biblioteca estática, disponibilizada em `libfinanceira.a`, permite incorporar o código necessário ao executável durante a ligação. Já a biblioteca dinâmica, gerada como `libfinanceira.so`, permite disponibilizar a implementação separadamente do programa cliente. Dessa forma, o projeto demonstra duas estratégias de vinculação e oferece alternativas de utilização da biblioteca.

A aplicação cliente complementa a biblioteca ao apresentar exemplos práticos de utilização das funções dos quatro módulos. Sua implementação permite observar como as interfaces públicas são utilizadas e como as diferentes funcionalidades podem ser reunidas em um único programa.

Outro resultado importante foi a construção de um `Makefile` completo, que centraliza as etapas de compilação e automatiza tarefas como a geração das bibliotecas, a construção dos executáveis, a execução da aplicação e dos testes e a limpeza dos arquivos gerados. O controle de dependências também permite realizar compilações incrementais, evitando recompilar arquivos que não precisam ser atualizados. Os alvos `run` e `run-static` possibilitam executar a aplicação cliente utilizando, respectivamente, as versões dinâmica e estática da biblioteca.

Além disso, foi incorporada a geração automática da documentação técnica por meio do Doxygen. A partir dos comentários do código-fonte, dos cabeçalhos e do arquivo `README.md`, são geradas páginas HTML que organizam as informações sobre as funções e suas interfaces. A documentação é disponibilizada em `docs/html/`, facilitando a consulta às funcionalidades e contribuindo para a manutenção e a utilização da biblioteca.

Por fim, os testes automatizados permitem verificar o comportamento das funções, incluindo resultados numéricos e condições de erro. A identificação e a correção de falhas durante o desenvolvimento, como o problema no expoente da função `juros_compostos`, demonstram a importância desse processo para o aperfeiçoamento da implementação e para a consistência da versão final entregue.

12 Referências bibliográficas

Referências

[1] FREE SOFTWARE FOUNDATION. *Using the GNU Compiler Collection (GCC).* Disponível em: `https://gcc.gnu.org/onlinedocs/`. Acesso em: 29 set. 2026.

[2] FREE SOFTWARE FOUNDATION. *GNU Make Manual.* Disponível em: `https://www.gnu.org/software/make/manual/`. Acesso em: 01 out. 2026.

[3] CHACON, Scott; STRAUB, Ben. *Pro Git.* Disponível em: `https://git-scm.com/book/pt-br/v2`. Acesso em: 02 out. 2026.

[4] ISO/IEC. *ISO/IEC 9899:2018: Programming Languages — C.* Geneva: International Organization for Standardization, 2018.

[5] *Trabalho Prático 2: Desenvolvimento de uma Biblioteca Modular em C.* Material de especificação fornecido no E-disciplinas